Day 1 Evidence Workbooklet — Instructor Key

Visible Output and Stored State

Engineering practice → computational evidence → Python output/state → support route

Instructor key. Same layout as student copy; solutions filled in response boxes. Print format: duplex, left-stapled packet.

Name		**Section**		**Date**	
Score	______/39		Mastery check	secure / developing / needs support	

Your team is doing a first-day hallway cart check. A small wheeled cart is rolled along a taped 12-meter path. One teammate times the run with a stopwatch. Another teammate enters the values in a Python notebook so the team can decide what should go into the lab record.

The problem today is not advanced physics. The problem is evidence: what did the notebook store, what did it visibly show, and which value is safe to write in the record?

Engineering task	Build a reliable evidence trail for two hallway cart trials before a result is copied into a lab record.
Computational move	Separate measurement inputs, stored variables, printed output, notebook display, and later overwrites.
Python focus	assignment, arithmetic expression, variable overwrite, `print`, final-expression notebook display, state after each cell.
Evidence to keep	exact visible lines, displayed values, stored values after each cell, a corrected lab note, and one non-mutating check.

Day 1 evidence rules

1. Assignment stores or overwrites a value. Example: `speed_mps = distance_m / time_s`.
2. `print(...)` creates visible output but does not by itself change a variable.
3. In a notebook, a bare expression at the end of a cell can display a value. Displaying a value is not the same as storing it.
4. A later assignment can overwrite an earlier stored value. Your trace must say what is stored *after each cell*, not just what looked important on the screen.

1 Read the notebook as an evidence system

recognize predict

Use the scenario above to identify what kind of evidence each line can create. Do not calculate everything yet.

```python
# Cell 1: first run on the hallway path
trial_id = "cart-A"
distance_m = 12.0
time_s = 3.0
speed_mps = distance_m / time_s
print("trial", trial_id)
print("speed", speed_mps)

# Cell 2: second run, same path, new time
trial_id = "cart-B"
time_s = 2.4
speed_mps = distance_m / time_s
print("trial", trial_id)
speed_mps + 0.2

# Cell 3: keep the adjustment separate
adjusted_speed = speed_mps + 0.2
print("stored", speed_mps)
print("adjusted", adjusted_speed)

# Cell 4: decide to record the adjusted value
speed_mps = adjusted_speed
print("recorded", speed_mps)
```

Pts.	Prompt	My evidence
1	Which variable names store measurement inputs rather than calculated results?	**Key:** `distance_m` and `time_s`. `trial_id` is a run label.
1	Which line first calculates a speed from the measured distance and time?	**Key:** `speed_mps = distance_m / time_s`
1	Which line in Cell 2 displays a candidate adjusted value without storing that adjustment?	**Key:** `speed_mps + 0.2`; no assignment stores it.
1	Which line in Cell 4 overwrites the stored value of `speed_mps`?	**Key:** `speed_mps = adjusted_speed`

2 Trace stored state after each cell

trace predict

Complete the state table. If a variable does not exist yet, write `not defined`. The first row has hints; the later rows require you to track overwrites.

Checkpoint	`trial_id`	`time_s`	`speed_mps`	`adjusted_speed`	Reason for any change
After Cell 1	`cart-A`	`3.0`	**Key:** `4.0`	`not defined`	**Key:** Initial measured speed: `12.0 / 3.0`.
After Cell 2	**Key:** `cart-B`	**Key:** `2.4`	**Key:** `5.0`	**Key:** not defined	**Key:** Trial/time overwritten; speed recalculated. Final expression displays `5.2` but does not store it.
After Cell 3	**Key:** `cart-B`	**Key:** `2.4`	**Key:** `5.0`	**Key:** `5.2`	**Key:** Adjustment stored separately; `speed_mps` remains measured speed.
After Cell 4	**Key:** `cart-B`	**Key:** `2.4`	**Key:** `5.2`	**Key:** `5.2`	**Key:** `speed_mps = adjusted_speed` overwrites the speed variable.

Pts.	Prompt	My response
2	At what checkpoint does `speed_mps` first become `5.0`? Explain the measurement change that caused it.	**Key:** After Cell 2; `time_s` changes to `2.4`, so `12.0 / 2.4 = 5.0`.
2	At what checkpoint does `speed_mps` first become `5.2`? Explain what line caused the overwrite.	**Key:** After Cell 4; `speed_mps = adjusted_speed`.

3 Separate printed output from notebook display

predict trace

Visible evidence can come from printed output or from a final notebook expression. Complete both columns. If a cell has no notebook display, write `none`.

Cell	Printed output lines from `print(...)`	Notebook display from final bare expression
Cell 1	**Key:** `trial cart-A`; `speed 4.0`	**Key:** none
Cell 2	**Key:** `trial cart-B`	**Key:** `5.2`
Cell 3	**Key:** `stored 5.0`; `adjusted 5.2`	**Key:** none
Cell 4	**Key:** `recorded 5.2`	**Key:** none

Common trap to check

A displayed value can look official because it appears on the screen. But a lab record should say whether the value was printed, displayed as a candidate calculation, or stored as the current value after an assignment.

Pts.	Prompt	My response
2	In Cell 2, why is the displayed `5.2` not yet the stored value of `speed_mps`?	**Key:** It is a bare expression result. No assignment stores it, so `speed_mps` remains `5.0`.
2	In Cell 3, why is it useful to print both `stored` and `adjusted`?	**Key:** It separates the current measured value from the adjusted candidate before overwrite.

4 Debug the lab record

debug explain test

A teammate writes this lab note after Cell 2:

Lab note to debug

Trial cart-B speed was recorded as 5.2 m/s because the notebook showed 5.2.

Pts.	Prompt	My response
2	What is accurate in the teammate's note?	**Key:** The notebook did show `5.2` after Cell 2.
2	What is inaccurate or incomplete in the teammate's note?	**Key:** After Cell 2, `5.2` was only displayed. It was not yet stored/recorded in `speed_mps`; stored `speed_mps` was `5.0`.
2	Rewrite the note so it separates measured speed, displayed candidate adjustment, and the value actually recorded after Cell 4.	**Key:** After Cell 2, cart-B measured speed was stored as `5.0` m/s and `5.2` was only displayed as an adjustment candidate. After Cell 4, `speed_mps` stored `5.2` m/s.

Pts.	Prompt	My response
1	Write one line you could run after Cell 2 to check the stored value of `speed_mps` without changing it.	**Key:** `print(speed_mps)` or bare `speed_mps`.
1	Write one line you could run after Cell 3 to check whether `adjusted_speed` exists.	**Key:** `print(adjusted_speed)` or bare `adjusted_speed`.
2	Before calculating speed, what measurement condition should the team check so the result is physically meaningful? Write it in plain language.	**Key:** Time must be positive/nonzero and correspond to the measured 12-meter path.

5 Compare two possible edits

debug | implement | explain

The team considers two ways to apply the 0.2 m/s adjustment. Both can show `5.2`; they do not leave the same evidence trail.

Version A: keep candidate separate

```
adjusted_speed = speed_mps + 0.2
print("stored", speed_mps)
print("adjusted", adjusted_speed)
```

Version B: overwrite immediately

```
speed_mps = speed_mps + 0.2
print("stored", speed_mps)
print("adjusted", speed_mps)
```

Pts.	Prompt	My response
1	In Version A, what is stored in `speed_mps` after the code runs?	**Key:** `5.0`.
1	In Version A, what is stored in `adjusted_speed` after the code runs?	**Key:** `5.2`.
1	In Version B, what is stored in `speed_mps` after the code runs?	**Key:** `5.2`; immediate overwrite.
2	Which version gives a better audit trail for a lab record? Explain your choice.	**Key:** Version A, because measured speed and adjusted candidate remain separate until the record decision.
2	If the goal is to record the adjusted value only after review, write one sentence explaining why Cell 4 is a safer place to overwrite `speed_mps`.	**Key:** Cell 4 overwrites only after both measured and adjusted values have been made visible.

6 Mastery check and support route

transfer explain

Use your own evidence from the workbooklet. Do not write a generic answer.

Pts.	Prompt	My response
2	What is one example from this notebook where the screen showed a value that was not yet the stored value that would be recorded?	**Key:** Cell 2 displayed `5.2`, but `speed_mps` was still `5.0`.
2	In one or two sentences, explain why engineers should preserve both visible output and stored-state evidence during early computational checks.	**Key:** Visible output shows what appeared on screen; stored-state evidence shows what the program will reuse or record. Keeping both prevents a displayed candidate from being mistaken for an approved/current result.

My mastery check

Mark the strongest statement that is true after this workbooklet.

☐	Secure: I can trace stored state and visible output separately, including a displayed value that is not stored yet.
☐	Developing: I can calculate some values, but I still need support separating display, print, assignment, and overwrite.
☐	Needs support: I need a smaller example before I can explain what Python stored after each cell.

My next support move

My issue is mostly: setup / Python syntax / tracing / arithmetic / notebook display / confidence / time.
The first place I got uncertain was: ____________________
My next small action is: ask a partner / ask instructor / rerun a cell / rebuild the trace table / try the recovery version.

Workbooklet target: I can build an evidence trail that separates measurement inputs, stored variable state, printed output, notebook display, and a later overwrite before a result enters an engineering record.